

Design → Designing s/w systems means determining how requirements are realized and result is SDD.

A SRS documents tells us "what" a system does and become input to the design process, which tells us "how" a s/w system works.

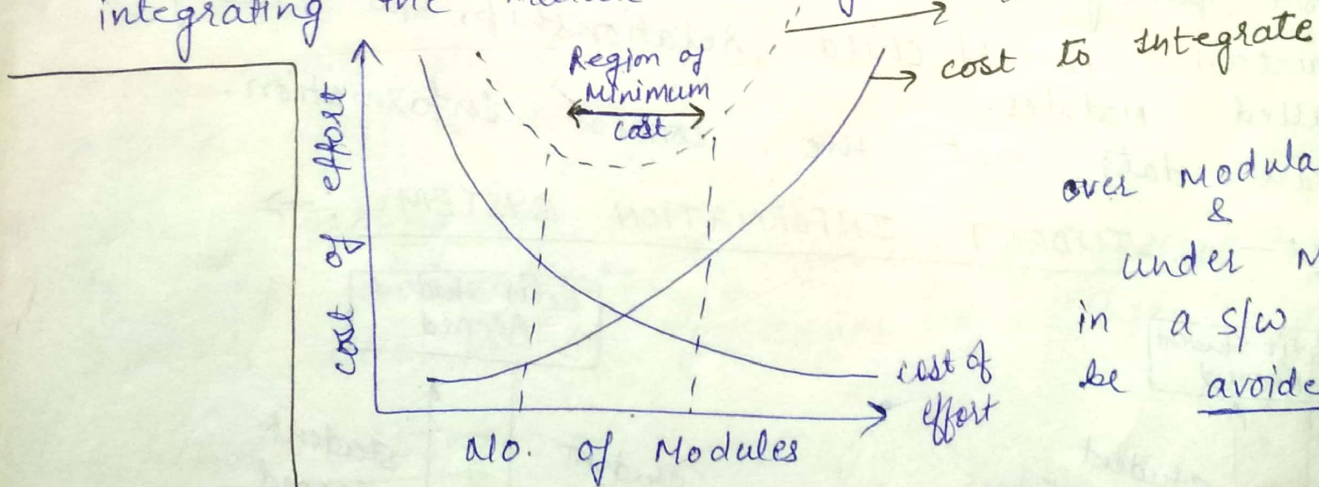
Design ← Conceptual Design
Technical Design

2.2 Modularity:-

- * Module range from "a module is an Ada Package" to "procedures and functions of PASCAL and C", to C++ / java classes".
- * A modular system consist of well defined, manageable units with well defined interfaces among the units.
- * A system is considered modular if it consists of discreet components such that each component can be implemented separately and a change to one component has minimal impact on other components.

To what Extent we shall Modularize:-

As the no. of modules grows, the effort associated with integrating the module also grows. Total s/w cost



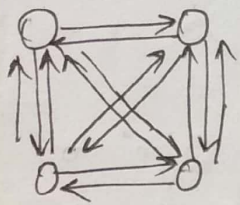
over modularity & under modularity in a s/w should be avoided.

2.2.1 Module Coupling :->

- * coupling is the measure of degree of interdependence between modules.

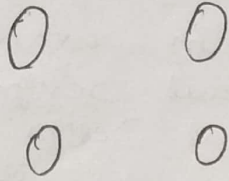
Two modules with high coupling are strongly dependent on each other.

- * Loosely coupled systems are made up of modules are relatively independent.
- * Highly coupled systems share a great of dependence b/w modules. (eg. share global variables).



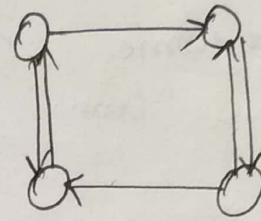
highly coupled (many dependencies)

(a)



uncoupled (No dependency)

(b)



loosely coupled (some dependency)

(c)

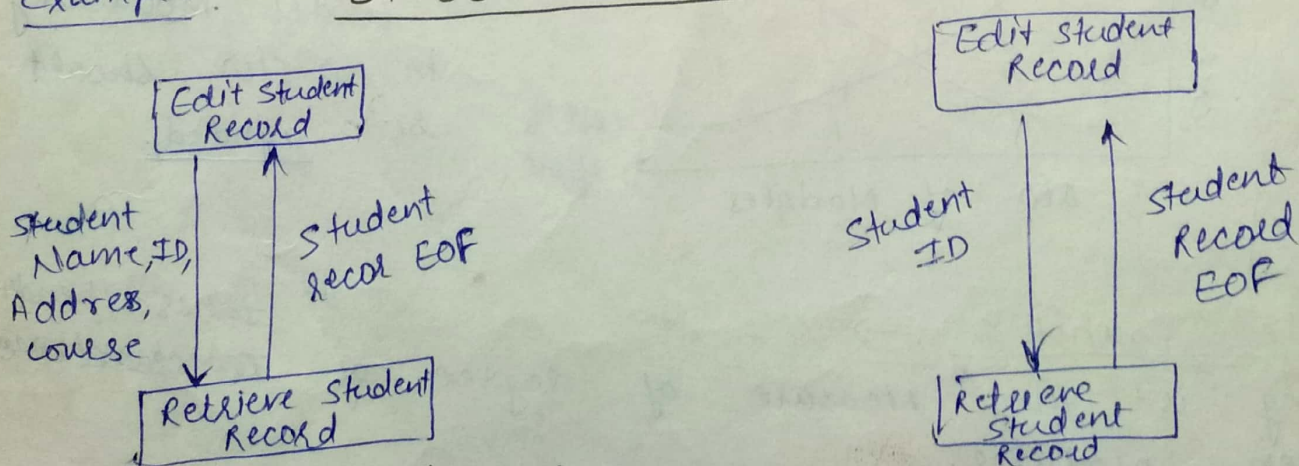
- * A Good design will have low coupling. Thus interfaces should be carefully specified in order to keep low value of coupling.

- * Coupling is measured by the no. of inter connections b/w modules. coupling increases as the no. of calls b/w modules increases, or amount of shared data increases.

Loose coupling can be achieved :-

1. Controlling the no. of parameters passed amongst modules.
2. Avoid passing undesired data
3. Maintain Parent/child relationship, b/w calling and called modules.
4. Pass data not the control information.

Example :- STUDENT INFORMATION SYSTEM :->



Poor Design: Tight Coupling

of Coupling :→

Data coupling —
 Stamp coupling —
 Control coupling —
 External coupling —
 Common coupling —
 Content coupling —

(low coupling)
 Best

↑ strength of coupling

Worst
 (highest)

1. Data coupling :→ The dependency b/w modules A and B is said to be data coupled if their dependency is based on the fact they communicate by only passing of data.

- * other than communicating through data, two modules are independent.
- * A good strategy is to ensure that no module communication contains "tramp data". In this way module dependency is minimized.

2. Stamp coupling :→

- * When complete data structure is passed from one module to another.
- * Stamp coupling typically involves tramp data.
- * If one procedure only needs a part of a data structure calling module should pass just that part, not the complete data structure.

3. Control coupling :→

If they communicate by passing of control information. This is accomplished by means of flags.

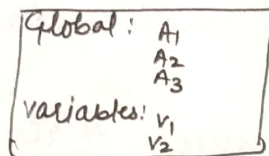
4. External coupling :→

A form of coupling, in which a module has a dependency to other module, external to the s/w being developed or a particular type of h/w.

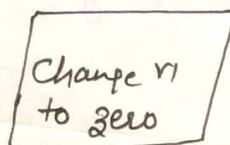
- * Communication to External tools and devices.

5. Common Coupling : →

- * With common coupling, Module A and B have data.
- * Global data areas are commonly found in program languages.
- * Making a change to the common data means tracing back to all the modules which access that data to evaluate the effect of change.
- * With common coupling it is difficult to determine which module is responsible for having set a variable to a particular value.



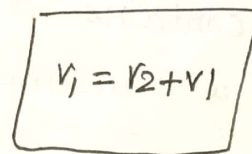
common data area and variable names



Module X



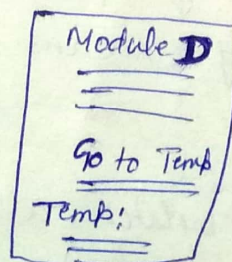
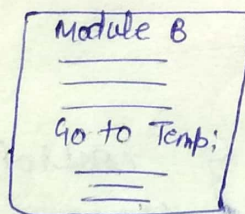
Module Y



Module Z

6. Content Coupling : →

It occurs when Module A changes data of Module B or when control is passed from one module to the middle of another.



Module Cohesion :->

Cohesion is a measure of the degree to which elements of a module are functionally related.

- * A strongly cohesive module implements functionality that is related to one feature of the solution and requires little or no interaction with other modules.
- * Cohesion may be viewed as a glue that keeps the module together.

An imp. design objective is to maximize the module cohesion and minimize the module coupling.

Types of Cohesion :-> various forms of cohesion b/w procedures X and Y.

- Functional cohesion
- Sequential "
- Communicational "
- Procedural "
- Temporal "
- Logical "
- Coincidental "

↑ high
↓ worst

1. Functional cohesion :-> X and Y are parts of a single functional task. Such a module often transforms a single input datum into single output datum.
eg - calculate current GPA & 'cumulative GPA'.
2. Sequential cohesion :-> X outputs some data which forms the input to Y.
eg. Addition of marks of individual subjects into a specific format is used to calculate GPA as input.
3. Communicational cohesion :-> X and Y both operate on the same data.

Procedural cohesion : $\rightarrow$ X and Y are both structured the same way. This is a poor reason for putting them in the same procedure.

* Procedural cohesion occurs in modules whose instructions although accomplish different tasks.

for eg $\rightarrow$ "calculate student GPA", print student record, calculate cumulative GPA.

Temporal cohesion : - X and Y both must perform around the same time. So, module exhibits temporal cohesion when it contains tasks that are related by the fact that all tasks must be executed in the same time-span. This is not a good reason to put them in the same procedure.

eg $\rightarrow$ set of functions responsible for initialization, start-up activities associated with programs exhibit temporal cohesion.

Logical cohesion : $\rightarrow$ X and Y perform logically similar operations. Logical cohesion occurs in modules that contain instructions that appear to be related because they fall into same logical class of functions.
 $\rightarrow$ Duplication can occur.

Coincidental cohesion : $\rightarrow$ X and Y have no conceptual relationship other than shared code.

Hence coincidental cohesion exists in modules that contain instructions that have little or no relationship to one another.

eg - one component is made with two unrelated parts.

for eg $\rightarrow$ check validity and print is a single component with two parts.

$\Rightarrow$ A good designer must design the modules with goal of high cohesion and low coupling.

Function Oriented Design $\rightarrow$ (2 levels)

Design

- $\rightarrow$ focus is on deciding which modules are needed
- $\rightarrow$ Specification of these modules, & how the modules should be interconnected.

Function Oriented Design $\rightarrow$ It is an approach to S/W design where the design is decomposed into a set of interacting units, where each unit has a clearly defined function. Thus, system is designed from a functional view.

-point.

(step-wise refinement) $\rightarrow$ (creator of PASCAL)

- $\rightarrow$ It is a top down design Method. $\rightarrow$ Niklaus Wirth
- $\rightarrow$ Starts with high level description of what the program does.
- $\rightarrow$ Then in each step, we take one part of our high-level description and refine it. i.e. specify in greater detail what that particular part does.

Example of scheme interpreter $\rightarrow$

```
while  
{
```

```
    Read an Expression from terminal;  
    Evaluate the "  
    Print the value;  
}
```

```
Print(Expression exp)
```

```
{
```

```
    Switch (exp  $\rightarrow$  type)
```

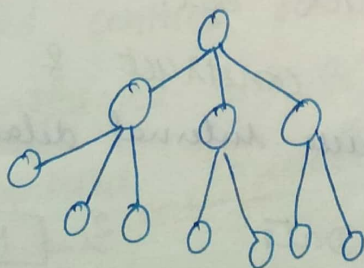
```
    case integer: /* Print the integer */
```

```
    case real: /* Print the real */
```

```
    case list:
```

```
    }
```

We continue the refinement of each module until we reach the statement level of our programming language.

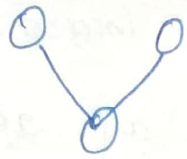


$\rightarrow$ Program will look like a tree.

Top-down structure

other module.

For a module to be reusable, however, we must require that several modules in design structure.



→ Design reusable structure.

Design Notations :→ Two things are important

1. Design
2. Design Process.

Design Notations are used during the design process, and are used to represent design or design decisions.

for function oriented design, design can be represented by:

- DFD
- Data Dictionaries
- structure charts
- Pseudocode

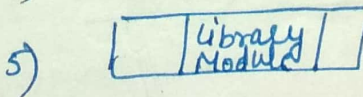
Structure charts :→

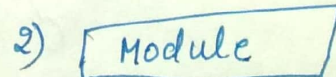
- * It partitions a system into black boxes.
- * Black box means that functionality is known to the user, without the knowledge of internal design.
- * Input is given to Black box and appropriate o/p are generated by black box.
- * systems are easy to construct & easy to maintain
- * reduces complexity because internal details are hidden

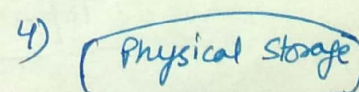
Notations :→

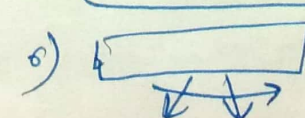
1)  Data

2)  Control

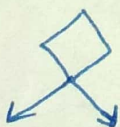
3)  Library Module

4)  Module

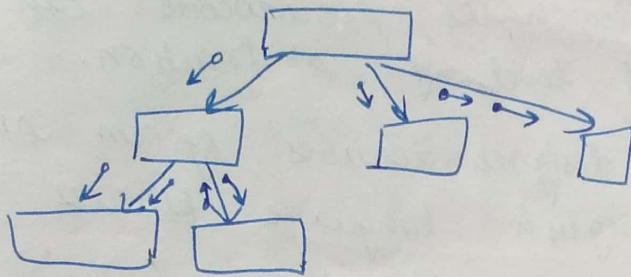
5)  Physical Storage

6)  Repetitive Call Module

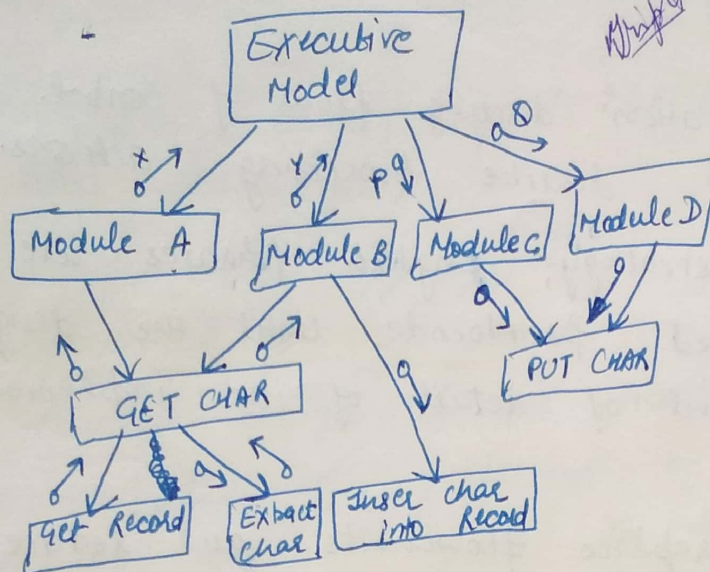
7)



for conditional module.

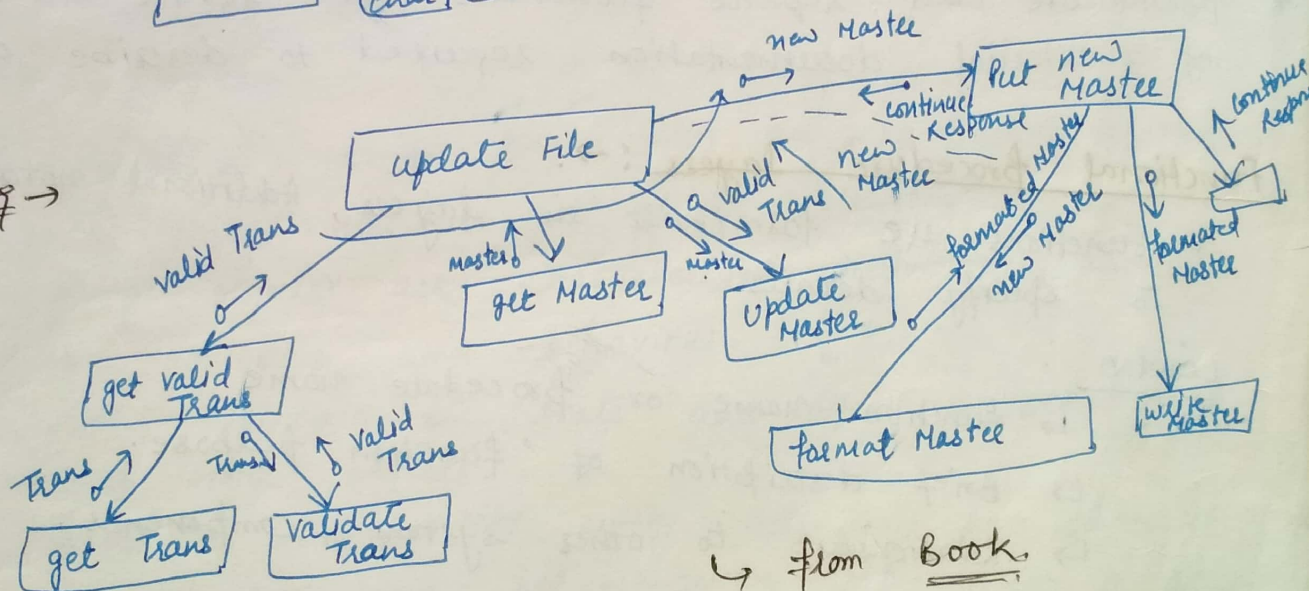


eg →



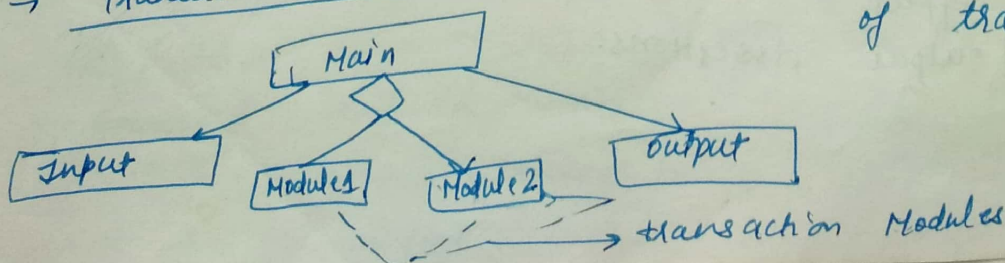
→ first draw the root module and then the modules that it invokes.

eg →



↳ from Book

- These are transformed centered structures.
- TCS (transformed centered structures) receives an input and it is transformed by a sequence of operations.
- Transaction centered structures : → describes a system that process different types of transactions



Pseudocode → Like flowcharts, pseudocode can be used at any desired level of abstraction.

- * Using pseudocode, the designer describes system characteristics using short, concise, English language phrases, that are structured by key-words such as 'if-Then-Else', 'while-Do', and 'End'.
- * keywords & indentation describe flow of control.
- * English phrases describe processing actions.
- * Using Top-down strategy, English phrases are expanded into more detailed pseudocode, until the design specification reaches the level of detail of the implementation language.
- * pseudocode can replace flowcharts and reduce the amount of external documentation required to describe a system.

Functional procedure layers : →.

- * Functions are formed/built in layers, Additional notations to specify details.

Level 0 :

- ↳ function name or procedure name
- ↳ Brief description of function purpose.
- ↳ Relationship to other system components.
- ↳ Author, date.

Level 1 :

- ↳ Function parameters (problem variables, types, purpose)
- ↳ Global variables (problem variables, types, purpose, sharing info).
- ↳ routines called by function.
- ↳ Side effect
- ↳ input/output Assertions.

- ↳ local data structure
- ↳ Timing constraints
- ↳ Exception handling (conditions, response, events).
- ↳ Any other limitation.

Level 3:

Body (structure chart, English pseudocode, decision tables, flow charts, etc.).

→ Object Oriented Design : →

↳ object oriented design is the result of focusing attention not on the function performed by the program but on the data that are to be manipulated by the program.

→ object oriented design begins with an examination of the real world 'things' that are part of the problem to be solved.

→ These things (objects) are characterized individually in terms of their attributes and behaviour.

→ Each object maintain its state and offers a set of services to other objects.

→ Each object is independent entity, that may be changed because all state and representation information is placed within the object itself.

→ objects can be distribute, and may execute sequentially or in parallel.

Basic Concepts : →

OOD (object oriented design) is not dependent on any implementation language. Problems are modelled using objects.

objects have :

* Behaviour (they do things)

* state (which changes when they do things).

e.g. car is an object.

State: → whether its engine is running

behaviour: → starting the car, which changes its state from "engine not running" to "engine running".

Terms related to object oriented design: →

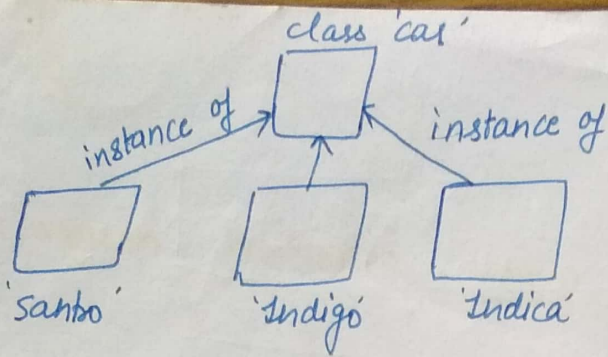
- 1) objects
- 2) Messages
- 3) Abstraction
- 4) classes
- 5) Attributes
- 6) operations
- 7) Inheritance
- 8) Polymorphism
- 9) Encapsulation.
- 10) Hierarchy.

1) objects → an entity ^(information) able to have a state and which offers a no. of operation. ^(behaviour)
unique identification & are distinguishable.
eg. Two bananas may be of same colour but still different.

2) Messages → objects communicate by message passing.
Message consists of the identity of the target object, name of the requested operation and any other operation needed to perform the function.

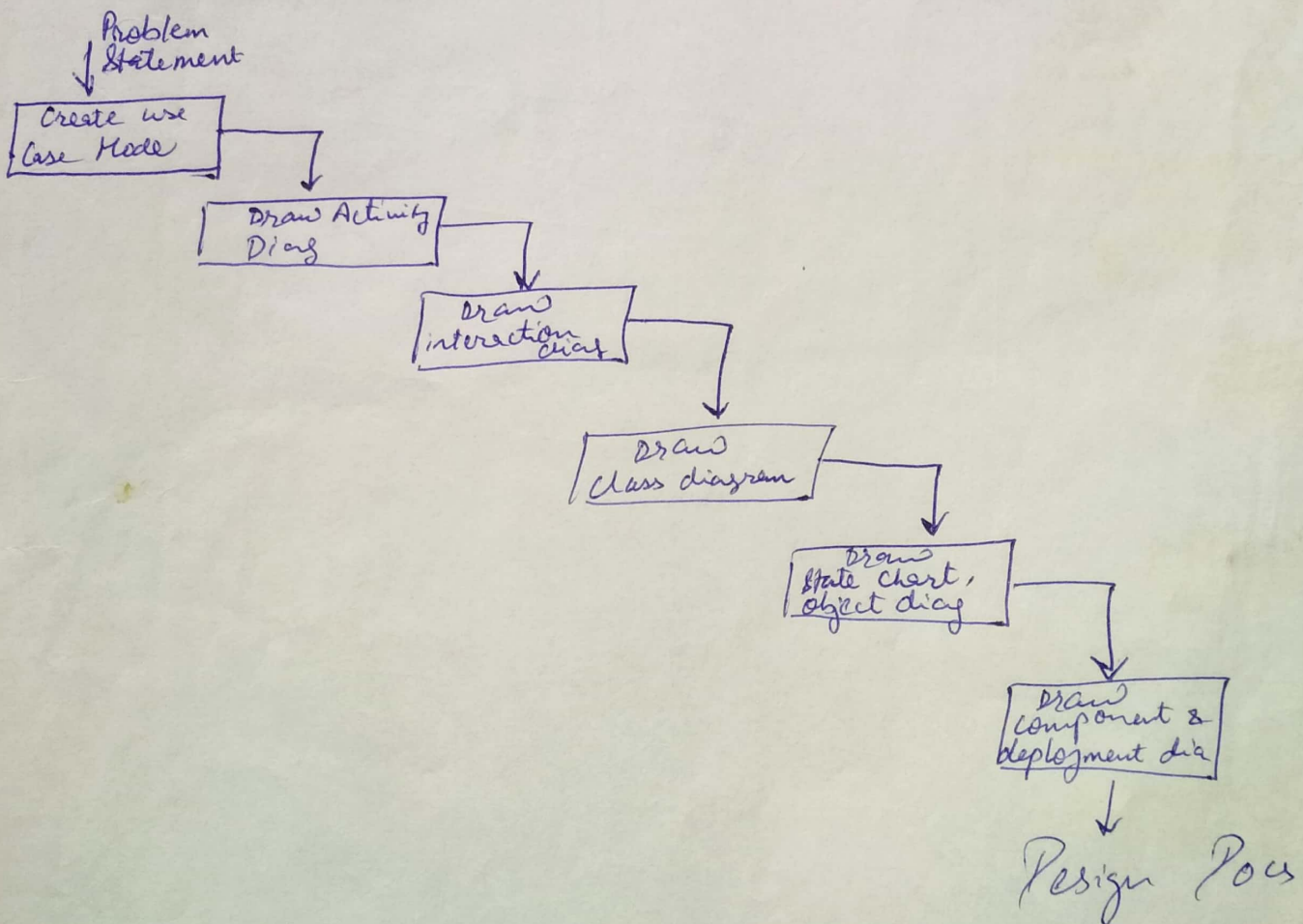
3) Abstraction: → complexity is managed using abstraction.
Abstraction is the elimination of the irrelevant and amplification of the essentials.

4) Class → In any system, there shall be number of objects.
Some of the objects may have common characteristics, and we can group ^{them} according to these characteristics.
This type of grouping is called a 'class'.
∴ Class is set of objects that share a common structure & a common behaviour.



class 'square'

Square	Name
colour	} Attribute
Points[4]	
see colour()	} <u>operations</u>
Draw()	



Steps for analysis & design of object oriented system.

